9/

Booth's Algorithm (Signed binary multiplication) / 2's complement multiplication.

Booth's Algorithm gives a procedure for multiplying binary integers in signed 2's complement representation in an effivient way following flowchart shows the flow of Booth's algorithm.

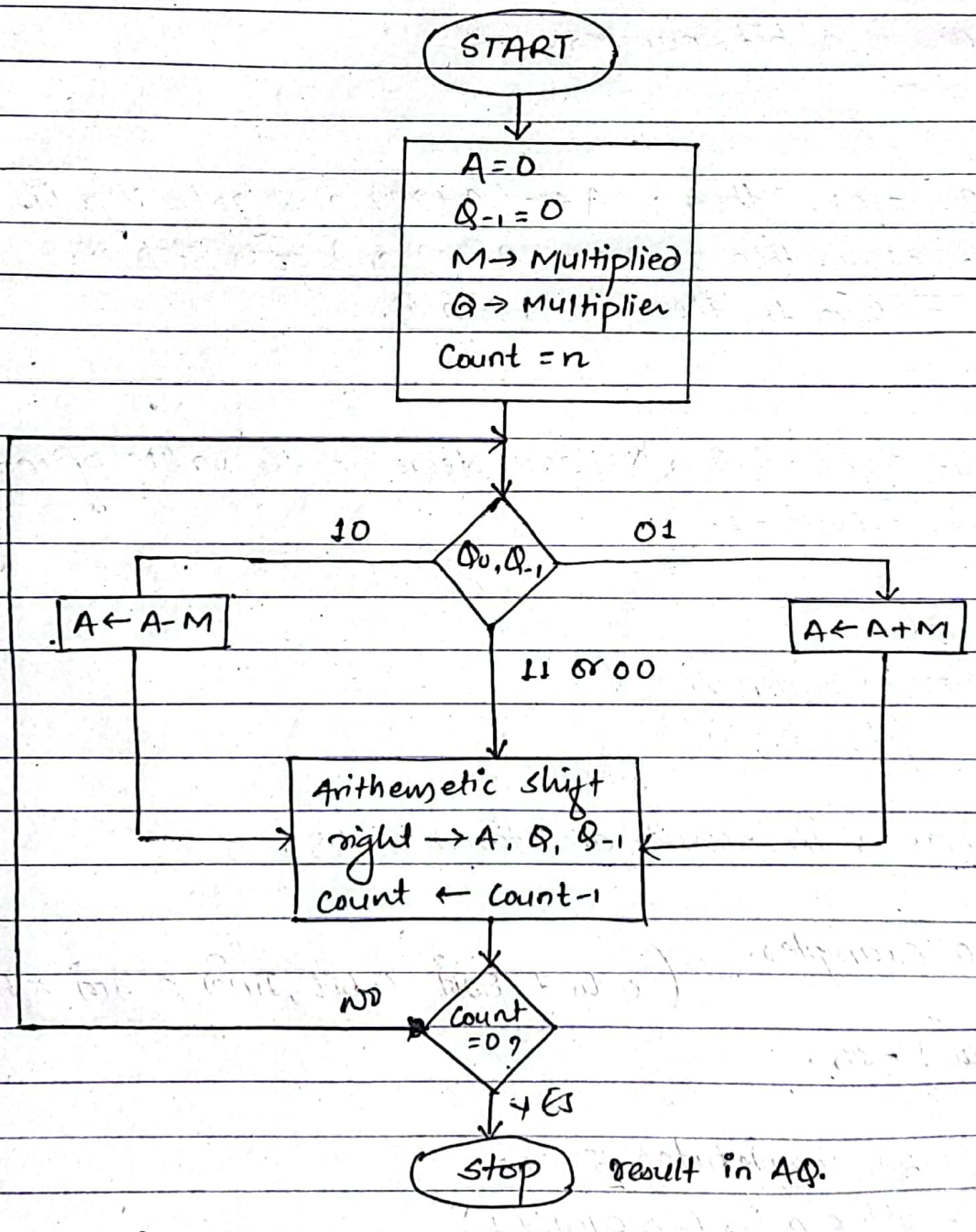


fig: flowchart of Booth's Algorithm.

Algorithm :-

i. initialize $A = 0$

$Q_1 = 0$.

Store the multiplicand in 'M' and multiplier in 'Q'.

Initialize the value of count of the no of bits involved

ie.

$Q = n$ if n bit number.

ii. check Q_0, Q_{-1}

ii.i - If $Q_0, Q_{-1} = 01$, then ; $A \leftarrow (A+M)$ and goto step iii.

ii.ii - If $Q_0, Q_{-1} = 10$, then ; $A \leftarrow (A-M)$ and goto step iii.

ii.iii. If $Q, Q_{-1} = 00$ or 11, then ; goto step iii.

iii. Arithemetic shift right A, Q, Q_{-1} and decrease the value of count 1

ie. count $\longleftarrow$ cont - 1.

iv. check of count = 0

iv.i If yes, goto step V.

iv.ii If NO, goto step ii.

v. stop the procedure : The results are stored in AQ.

● Booths Algorithm Examples. (0 to 7 सम्म 4 bit, माथि 8 भए 5 bit हुन्छ)

Q. multiply -5 and -15.

now;

$M = -5 = 2's$ complement of 5 +1

$= 1^{st}$ cmp of $(-0\,0101 + 1)$

$= 11010 + 1 = 11011$

Q = -15 = 2's complement of 01111
= 1st complement of 01111 + 1
= 10000 + 1
= 10001

-M = 1's complement of M + 1
= 00100 + 1
= 00101

M = 11011
-M = 00101
Q = 10001 Cout

now, let A = 00000 Q-1 = 0, ~~count~~ = 5 then proceeding Booth's algorith

	A	Q	Q-1	Count	Remarks.
i.	00000	10001	0	5	initialization
ii.	00101	10001	0	5	A ← A-M
	00010	11000	1	4	ASR AQ Q-1
iii.	11101	11000	1	4	A ← A+M
	11110	11100	0	3	ASR AQ Q-1
iv.	11111	01110	0	2	ASR AQ Q-1
v.	11111	10111	0	1	ASR AQ Q-1
vi.	00100	10111	0	1	A ← A-M
	00010	01011	1	0	ASR AQ Q-1

ANS = AQ = 01001011

$= 2^6 + 2^3 + 2^1 + 2^0$

$= 64 + 8 + 2 + 1$

$= 75$

Also, $-15 * (5) = 75$

Q. $3 * -7$

$M = 3$ 0011 $\quad -M = 1100 + 1 = 1101$

$Q = -7$ = 2's complement 0111.

$= 1000 + 1$

$= 1001$ //

A	Q	Q-1	Count	Remark.
i. 0000	1001	0	4	initialization.
ii. 1101	1001	0	4	$A \leftarrow A - M$
1110	1100	1	3	ASR
iii. 0001	1100	1	-3	$A \leftarrow A + M$
0000	1110	0	2	ASR
iv. 0000	0111	0	1	ASR
v. 1101	0111	0	1	$A \leftarrow A - M$
1110	1011	1	0	ASR

AQ = 11101011 $= -2^7 + 2^6 + 2^5 + 2^3 + 2^1 + 2^0$

$= -21$ //

Q. multiply (-5) x (-15) using Booth's algorithm.

soln Here,

$M = -5$ = 2's complement of 5

= 2's complement of (00101)

= 1's complement of (00101) + 1

= 11010 + 1

= 11011

$\therefore -M$ = 2's complement of M

= 2's complement of (11011)

= 1's complement of (11011) + 1

= 00100 + 1

= 00101

[Hint: No. of bit choose गर्दा सर्वप्रथम highest number लाई जति bit चाहिन्छ त्यसको 1 बढी bit लिने]

Also,

$Q = -15$

= 2's complement of 15

= 2's complement of (01111)

= 1's complement of (01111) + 1

= 10000 + 1 = 10001

from Booth's flow chart.

Let.

A = 00000

$Q_{-1} = 0$

and.

Count = 5

	A	Q	Q_{-1}	Count	Remark.
i.	00000	10001	0	5	Initialization.
ii.	00101 → 00010	10001 → 11000	0 → 1	5 → 4	A ← A−M ASR
iii.	11101 → 11110	11000 → 11100	1 → 0	4 → 3	A ← A+M ASR
iv.	11111	01110	0	2	ASR
v.	11111	10111	0	1	ASR
vi.	11010 → 11101	10111 → 01011	0 → 1	1 → 0	A ← A+M ASR
vii.	00100 → 00010	10111 → 01011	0 → 1	1 → 0	A ← A−M ASR.

Now, we get the result in AQ.

i.e.

$$AQ = 00010\ 01011$$
$$= 2^6 + 2^3 + 2^1 + 2^0$$
$$= 64 + 8 + 2 + 1$$
$$= 74 + 1$$
$$= 75$$

correct answer #.

(Division of unsigned Binary integers)

- Restore Division Algorithm.

Restore division algorithm gives a procedure for dividing unsig integers in an efficient way.

following flowchart shows the flow of restore division algorithm

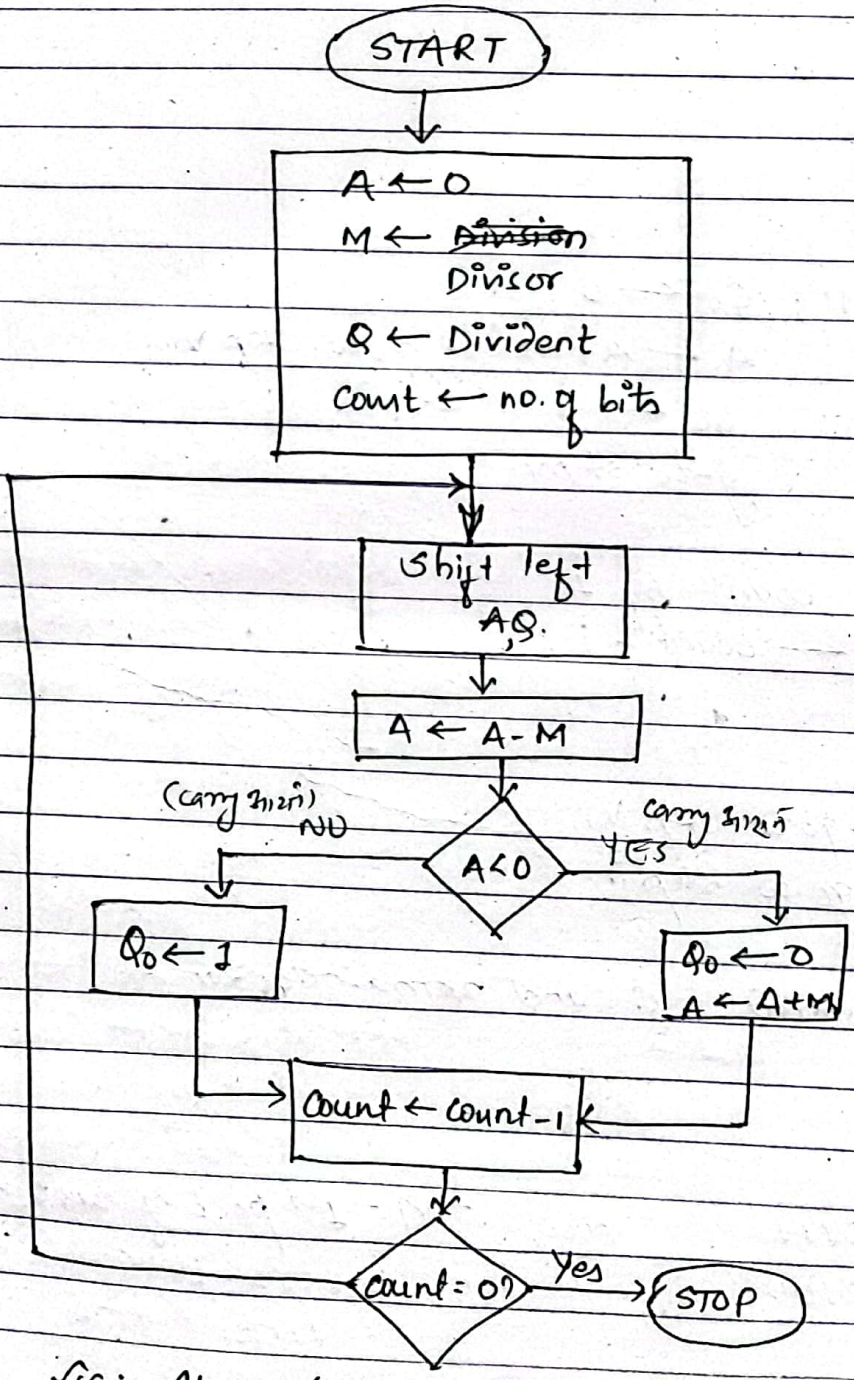


fig: flow chart of Restore division Algorithm.

Algorithm :-

i. Initialize $A=0$
store divisor as 'M' and divident as 'Q'
Initialize the value of count as no of bits involved.
ie. count = n if n bit number

ii. shift left A, Q.

iii. perform $A \leftarrow A-M$

iv. check if $A < 0$

iv.i. If yes ; $Q_0 \leftarrow 0$
$A \leftarrow A+M$, goto step v

iv.ii. If no ; $Q_0 \leftarrow 1$
goto step v.

v. decrease the value of count by 1
ie. count $\leftarrow$ count - 1

vi. check if count = 0

vi.i if yes; goto step vii.

vi.ii. if no ; go to step ii.

vii. STOP ; quotient is stored in Q and remainder in A.

eg: Perform (7/4)
Here,

Divident $Q = 7 = 0111$

Divisor, $M = 4 = 0100$

now; $-M$ = 1st part of $0100+1$
$= 1011+1$
$= 1100$

0111
+1100
(1)0011

0011
1100
1111

0000
+1100
A←A-M (1100)
↳ carry આવેલ

now, let A = 0000 , n = 4

now performing Restore division algorithm.

	A	Q	Count	Q_0	Remark.
i.	0000	0111 (Q_0)	4	1	initialization
ii.	0000	111-	4	-	shift left AQ
	1100	111-		0	A ← A-M
	1100	1110		0	A<0 true so; Q_0=0
	0000	1110	3		A ← A+M, count ← count -1
iii.	0001	110-	3		shift left AQ
	1101	110-			A ← A-M
	1101	1100		0	A<0 true so; Q_0=0
	0001	1100	2		A ← A+M ; count ← count -1
iv.	0011	100-	2		shift left AQ
	1111	100-		0	A ← A-M
	1111	1000		0	A<0 , true so; Q_0=0
	0011	1000	1		A ← A+M , count ← count -1
v.	0111	000-	1		shift left AQ
	0011	000-			A ← A-M
	0011	000-		1	A<0, not true so;
	0011	0001	0		Q_0 ← 1 ↳ count ← count -1

A←A-M
∴ 0111
+1100
1)0011

carry આવ્યો, so 1 ગયો. → Q

so; Quotient (Q) = 0001 = 1

Remainder (A) = 0011 = 3 //

8. perform 11/3.

Soln Here,

Q = 11 = 01011

M = 3 = 00011

$-M = 11100 + 1$

$= 11101$

Let A = 00000, n = 5 then;

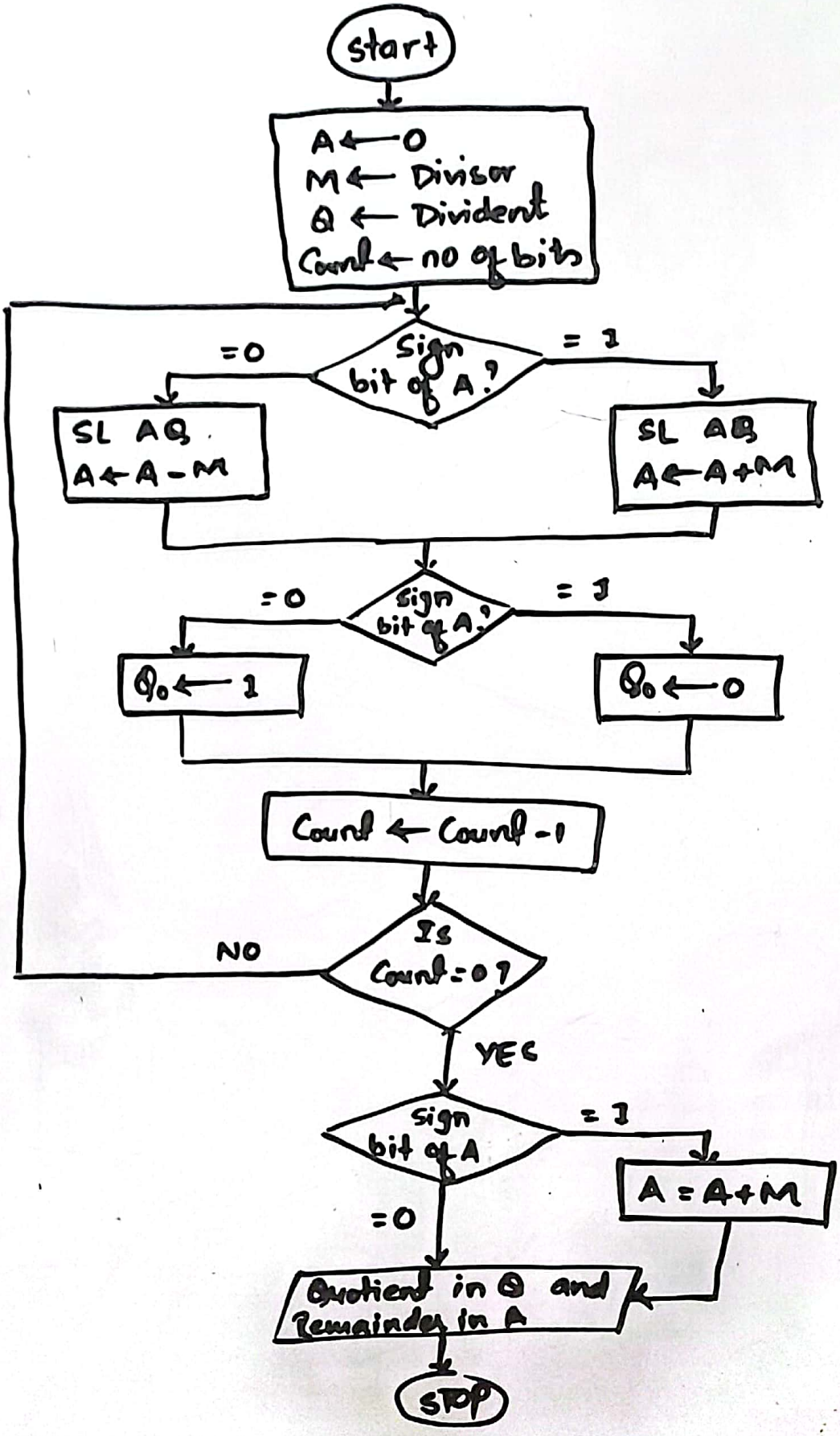
	A	Q	Count	Q_0	Remarks
i.	00000	01011	5	1	Initialization
ii.	00000	1011 –	5		
	01101	1011 –			ASL A, Q
	11101	1011 –			A ← A − M
	00000	1011 0		0	A < 0 so; $Q_0 \leftarrow 0$
			4		A < A+M ; Count ← Count−1
iii.	00001	011 0 –	4		
	11110	011 0 –			ASL A, Q
	11110	1100			A ← A − M
	00001	1100	3	0	A < 0, so; $Q_0 \leftarrow 0$
					A ← A+M ; Count ← Count−1
iv.	00010	1100 –	3		
	11111	1100 –			ASL A, Q
	11111	11000			A ← A − M
	00010	11000	2	0	A ≥ 0 so; $Q_0 \leftarrow 0$
					A ← A+M, Count ← Count−1
v.	00101	1000 –	2		
	00010	1000 –			ASL A, Q
	00010	10001			A ← A − M
	00010	10001	1	1	A ≥ 0 so; $Q_0 \leftarrow 1$
					Count ← Count−1

vi.	00101	0001-	1	ASL · AQ
	00010	0001-		A ← A - M
	00010	00011	1	A > 0 so; Q ← 1
	00100	00011	0	Count ← Count - 1

So; Quotient (Q) = 00011 = 3

Rem (A) = 00010 = 2 //

Non Restore Division Algorithm



(+15/-4) using Non-Restoring Division method.

sol^n Q = +15 = 01111
M = -4 = 11011+1 = 11100
∴ M = 00011+1 = 00100

A = 00000
Count = 5

	A	Q	Q_0	Count	Remark
i.	00000	01111	1	5	Initialization
ii.	00000	1111-	-	5	SL AQ
	00100	1111-	-	5	A ← A-M
	00100	11111	1	5	Q_0 ← 1
	00100	11111	1	4	Count ← count-1
iii.	01001	1111-	-	4	SL AQ
	01101	1111-	-	4	A ← A-M
	01001	11111	1	4	Q_0 ← 1
	01101	11111	1	3	Count ← Count-1
iv.	11011	1111-	-	3	SL AQ
	11011	1111-	-	3	A ← A-M
	11011	11110	0	2	Q_0 ← 0 Count ← Count-1
v.	10111	1110-	-	2	SLAQ
	10011	1110-	-	2	A ← A+M
	10011	11100	0	1	Q_0 ← 0 Count ← Count-1
vi.	00111	1100-	-	1	SLAQ
	00011	1100-	-	1	A ← A+M
	00011	11001	1	0	Q_0 ← 1 Count ← Count-1

Remainder (A) = 00011 = 3
Quotient (Q) = 11001

(11/3) using Non-Restoring Division method.

sgn Q = 11 = 01011 A = 00000
M = 3 = 00011 Count = 5
-M = 11100+1 = 11101

	A	Q	Q_0	Count	Remark
i.	00000	01011	1	5	Initialization
ii.	00000	1011-	-	5	SLAQ
	11101	1011-	-	5	A ← A - M
	11101	10110	0	4	Q_0 ← 0 Count ← Count - 1
iii.	11011	0110-	-	4	SLAQ
	11110	0110-	-	4	A ← A + M
	11110	01100	0	3	Q_0 ← 0, Count ← Count - 1
iv.	11100	1100-	-	3	SLAQ
	11111	1100-	-	3	A ← A + M
	11111	11000	0	2	Q_0 ← 0, Count ← Count - 1
v.	11111	1000-	-	2	SLAQ
	00010	1000-	-	2	A ← A + M
	00010	10001	1	1	Q_0 ← 1, Count ← Count - 1
vi.	00101	0001-	-	1	SLAQ
	00010	0001-	-	1	A ← A - M
	00010	00011	1	0	Q_0 ← 1, Count = Count - 1

Quotient (Q) = 00011 = 3
Remainder (R) = A = 00010 = 2

Lookup table:

- In computer programming, a look up table, also known as a LUT, is an array that holds values which would otherwise need to be calculated. The table may be manually populated when the program is written, or the program may populate the table with values as it calculates them. When the values are needed later, the program can look them up, saving CPU resources.
- Any combinational circuit can be implemented by ROM is a lookup table.
- The inputs to the combinational circuit serves as the address inputs of the ROM.
- The data o/ps of the ROM correspond to the o/ps of the combinational circuit.
- The ROM is programmed with data such that the correct values are o/p for any possible input values.
- Consider a 4x1 ROM programmed to mimic a two input AND gate

as AND gate

Address		Data
x	y	
0	0	0
0	1	0
1	0	0
1	1	1

↳ Lookup ROM equivalent.

- The i/p to AND gate serve as the address i/p to ROM and the o/p of ROM corresponds to AND gate o/p.
- By programming ROM with the data shown, it outputs the same values as the AND gate for all possible values of x and y.

2. Wallace Trees:

→ Wallace Tree is a combinational circuit used to multiply two numbers.

→ Although it requires more hardware than shift-add multipliers, it produces a product in less time.

→ Wallace trees use carry-save address and only one parallel adder.

→ A carry save adder can add three values simultaneously instead of just two ie. it outputs both : sum and carry.

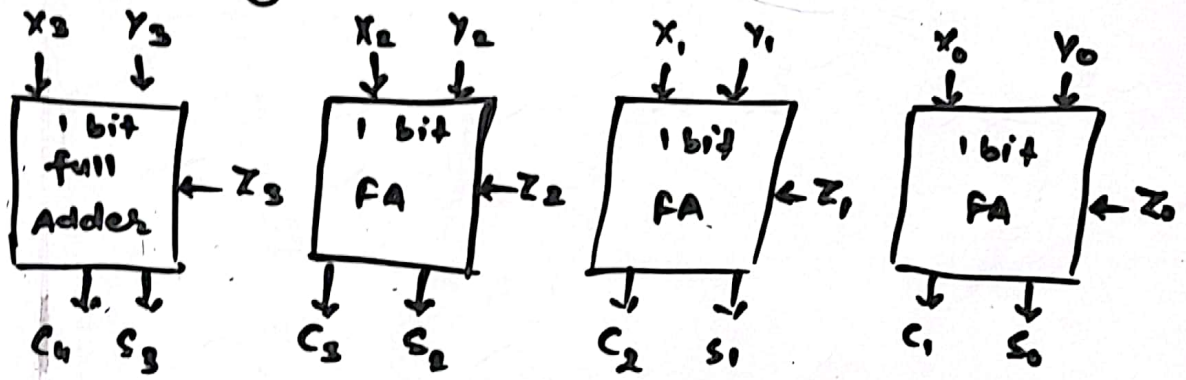


fig: A carry save adder.

→ for eg: let X = 0111 , Y = 1011 , Z = 0010

In such case sum will be of 4 bit and Carry will be of five bits, because c appears to be shifted one position to the left of Sums since adder that generates S_i also C_{i+1}.

→ To use carry save adder to perform multiplication, we first calculate the partial products of the multiplication then ilp them to the carry-save adder.

eg:

```
X = 111
Y = 110
-------
  000     ← pp0
 111      ← pp1
111       ← pp2
-------
101010    ← final sum calculated.
```

2. Wallace Trees:

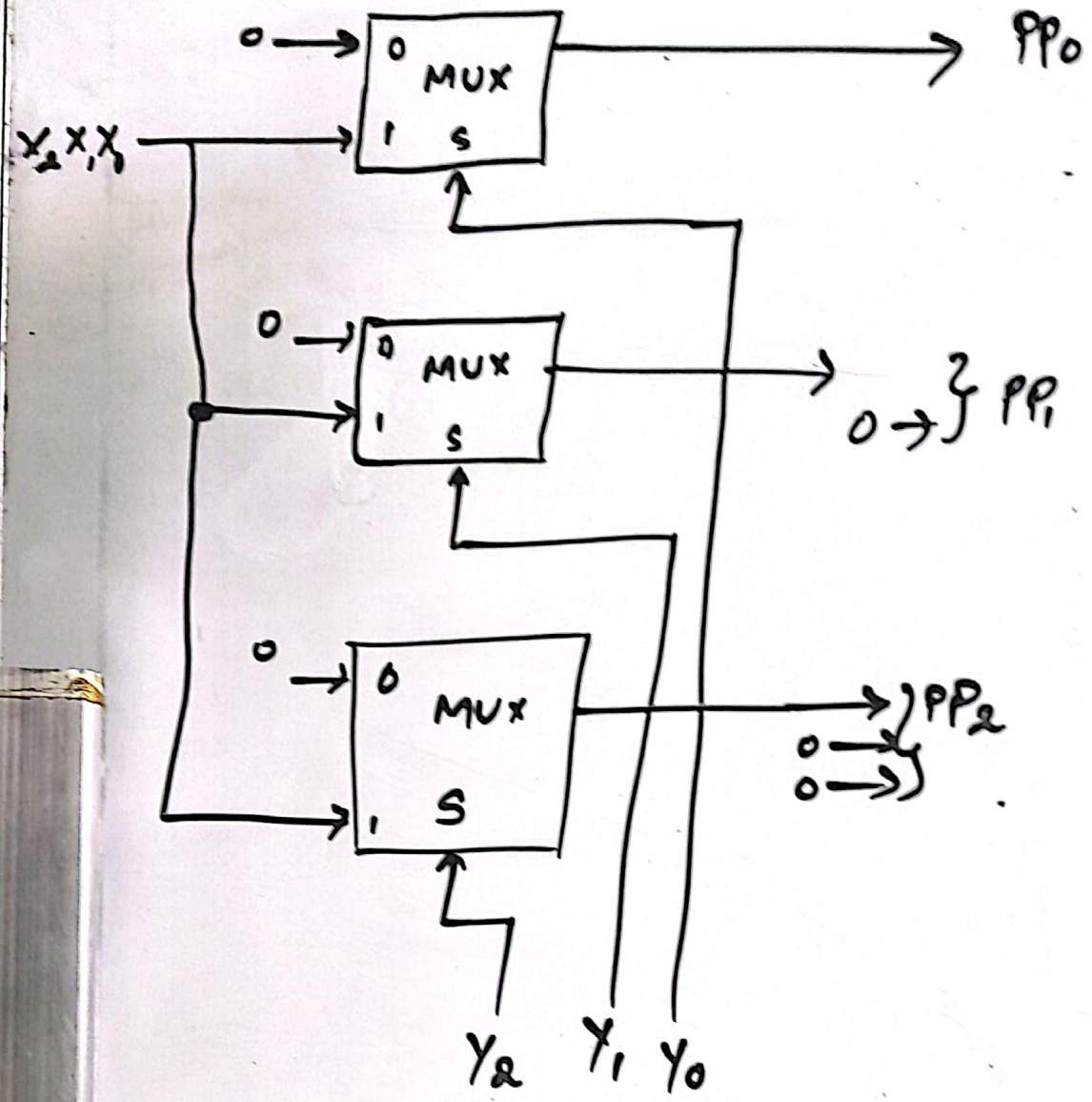


fig: Generating partial products for multiplication using wallace Tree.

→ Here,

$PP_1 = 1110$

$PP_2 = 11100$

#. 4x4 Wallace Tree Multiplier.

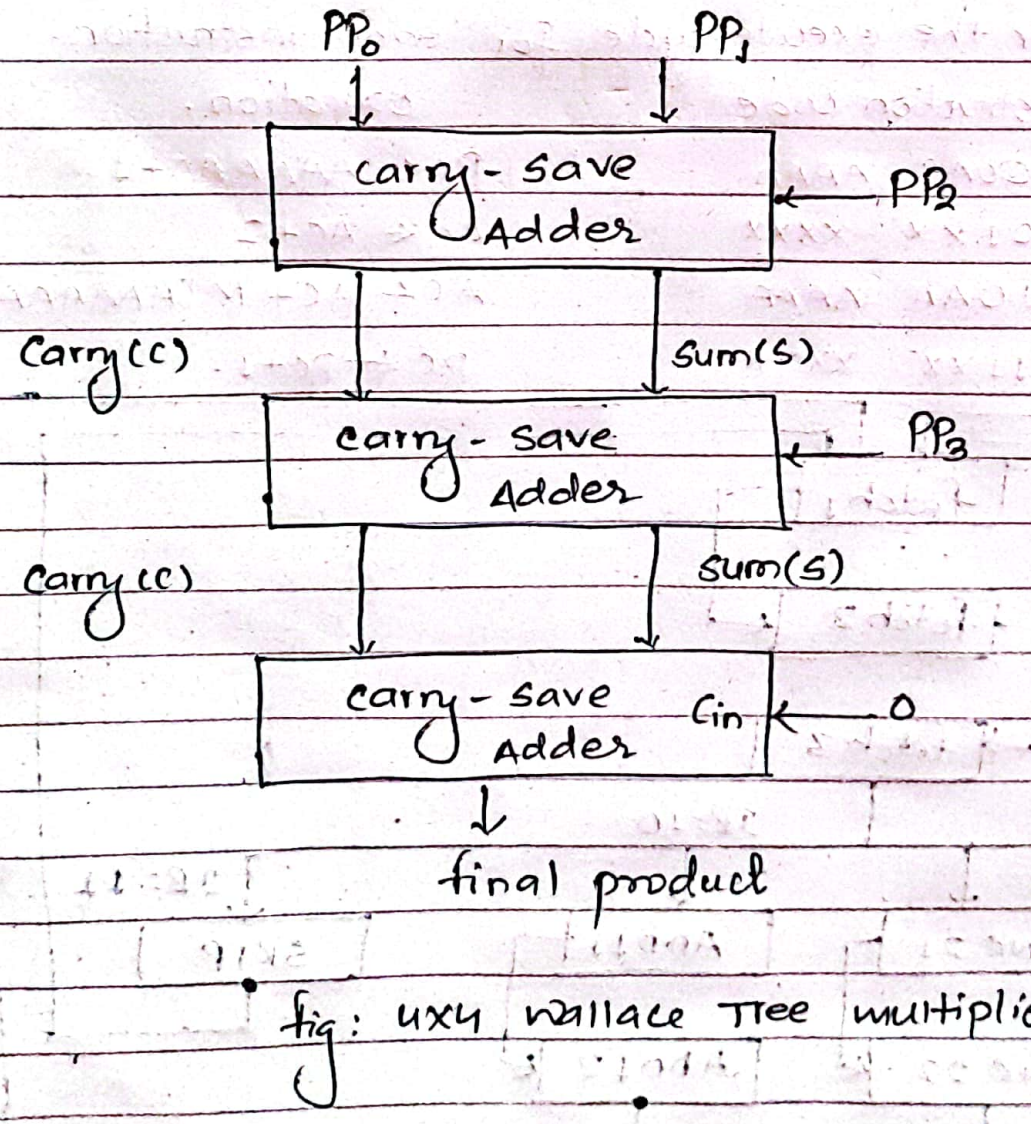


fig: 4x4 wallace tree multiplier.

8x8 wallace Tree multiplier

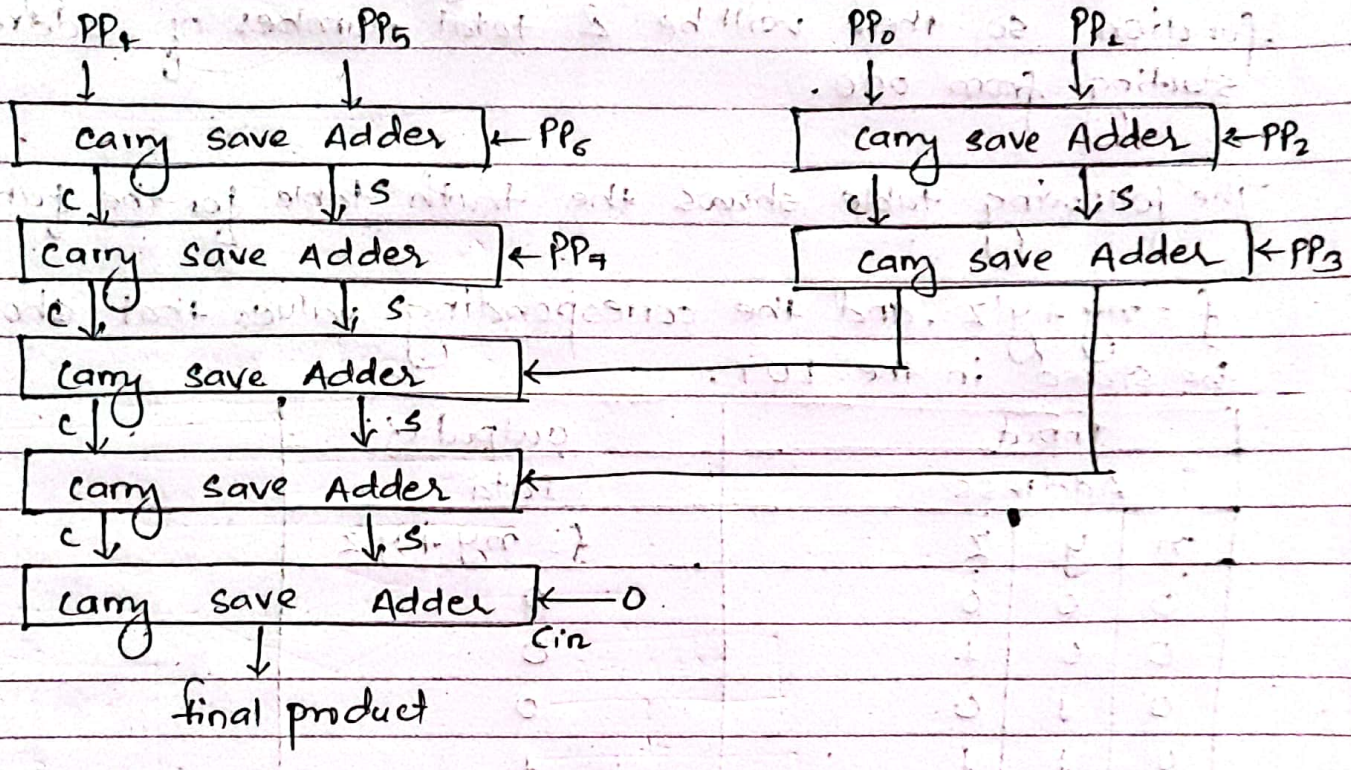


fig: 8x8 wallace Tree multiplier.

2019 spring: 4(b): What is Lookup ROM? Design a Lookup ROM to implement the function $f = xy + y'z$.

ans:

Lookup ROM: A Lookup ROM (LUT) is a type of read-only memory (ROM) that is used to store a table of values. The LUT can be used to quickly look up the value of a fundion for given set of input values.

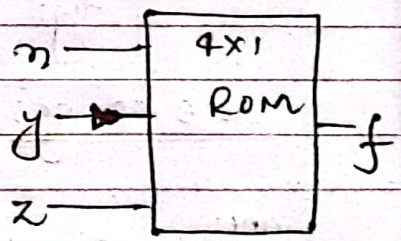
Here,

we have to design a Lookup ROM to implement the function $f = xy + y'z$

Since there are 3 variables (x, y and z) in the given function so there will be 8 total number of address starting from 000.

The following table shows the truth table for the function $f = xy + y'z$, and the corresponding values that should be stored in the LUT:

Input Address			output (f) Data
x	y	z	$f = xy + y'z$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



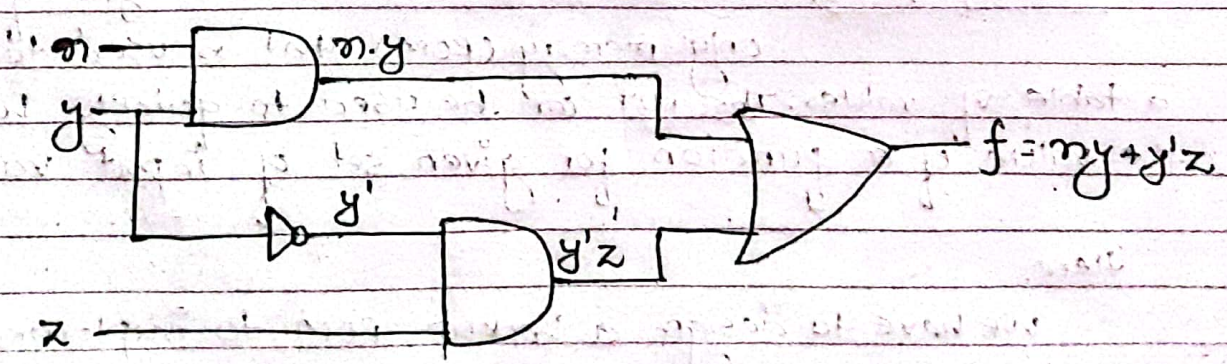


fig: Lookup ROM equivalent.

2018 Spring: How is floating point number represented in computer?
2019 (short)

IEEE 754 Standard (Floating Point Representation)

The most important floating point representation is defined in IEEE standand 754 which was adopted in 1985 and was ~~receive~~ revised in 2008. This standard was developed to facilitate the ~~prop~~ portability of sophisticated numerical operations from one processor to another. It covers both binary as well as decimal floating point representations.

IEEE 754-2008 defines following different types of floating point representation.

i. **Arithmetic format**:- All the mandatory operations defined by the standard are supported by this format. It is used to represent floating point operands and results of operation.

ii. **Basic format**:- This format cover's 5 floating point represent 3 binary and 2 decimals whose encodings are specified by the standard and can be used for arithmetic.

iii. **Interchange format**:- This format provides encodings that are suitable for data interchange between different platforms and for storage.

The floating point representation of a number has 2 parts. The 1st part represents fixed point number's called mantissa and the 2nd part represents position of decimal and is called exponent.

Eg:

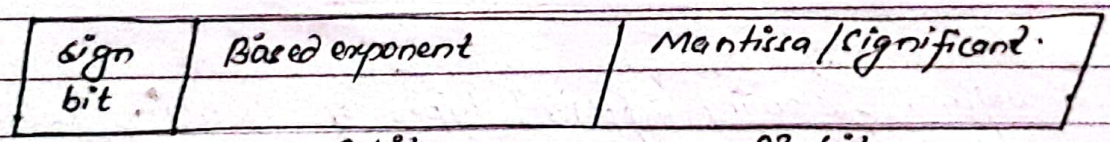
$0.1001110 * 2^{100}$

Here;

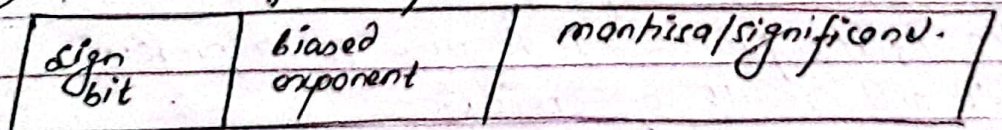
1001110 is the mantissa and 100 is the exponent.

The 8 basic formats have bit lengths of 32, 64 and 128 bits with exponents of 8, 11 and 15 bits.

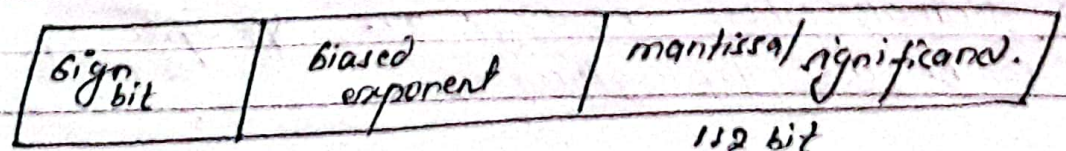
The standard formats for floating point representation are shown below:-



(a) Binary 32 format. (Single Format)



(b) binary 64 format. (Double format)



(c) binary 128 format

fig: IEEE 754 standard formats

Imp

Shift Add multiplication Method

flow chart:

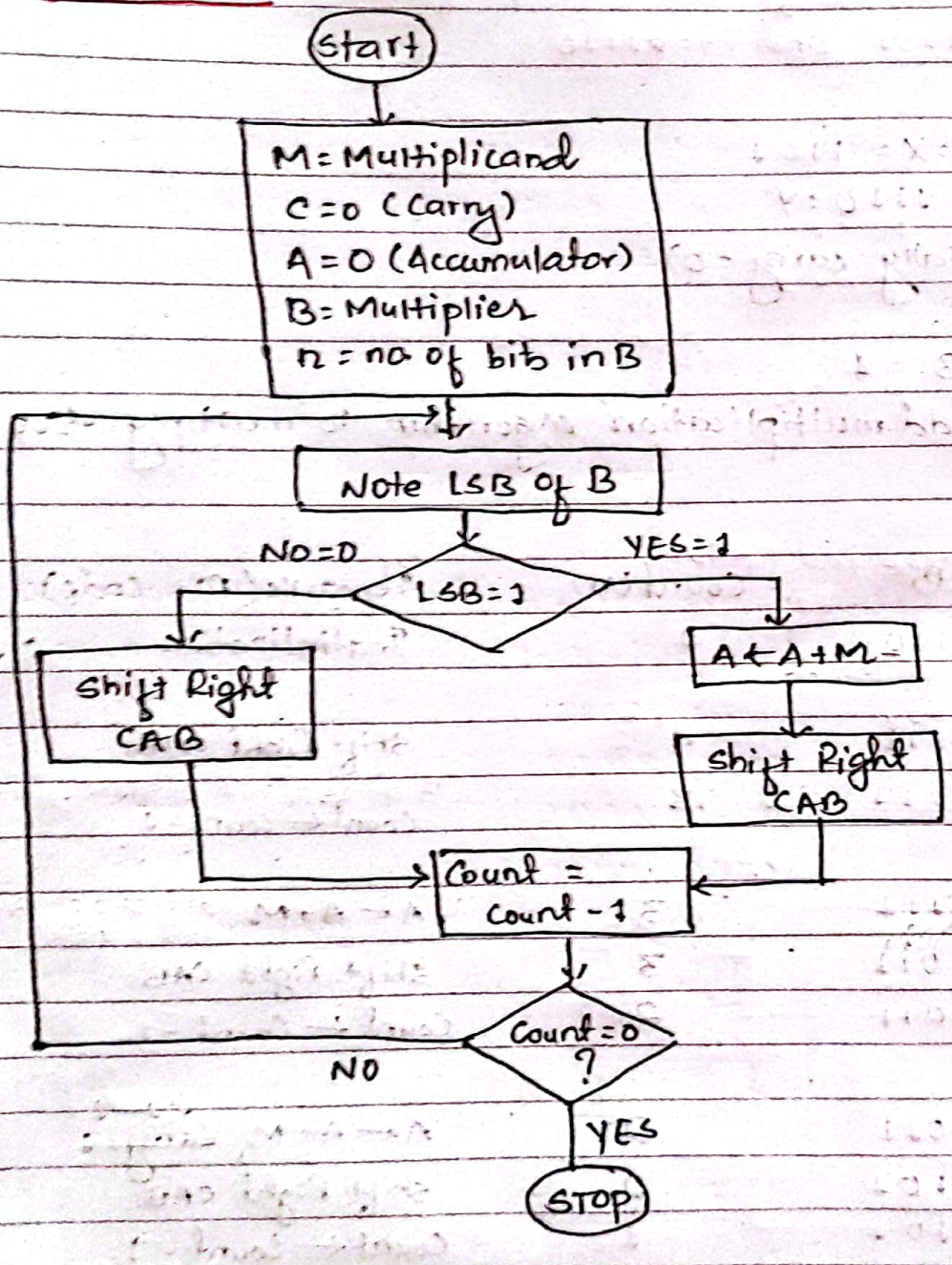


fig: Flow chart of shift add multiplication.

5. Trace the RTL code for shift add multiplication algorithm for X = 1101 and Y = 0110

Soln Here, Let,

Multiplicand (M) = X = 1101

Multiplier (B) = 1110 = Y

Carry (C) = 0 (initially carry = 0)

A = 0000

n = no of bits in B = 4

Now, applying shift add multiplication algorithm to multiply two numbers:

	C	A	B	Count(n)	Remarks (RTL code)
i.	0	0000	1110 (LSB bit)	4	Initialization
ii.	0	0000	0111	4	Shift Right CAB
	0	0000	0111	3	(B का LSB bit 0 होगया) Count ← Count - 1
iii.	0	1101	0111	3	A ← A+M (2nd step में B का LSB 1 होगया)
	0	0110	1011	3	Shift Right CAB
	0	0110	1011	2	Count ← Count - 1
iv.	1	0011	1011	2	A ← A+M, C ← 1
	1	1001	1101	2	Shift Right CAB
	1	1001	1101	1	Count ← Count - 1
v.	1	0110	1101	1	A ← A+M, C ← 1
	1	1011	0110	1	Shift Right CAB
	1	1011	0110	0	Count ← Count - 1

Result in AB = $\frac{1011}{A}\frac{0110}{B}$

checking Result:

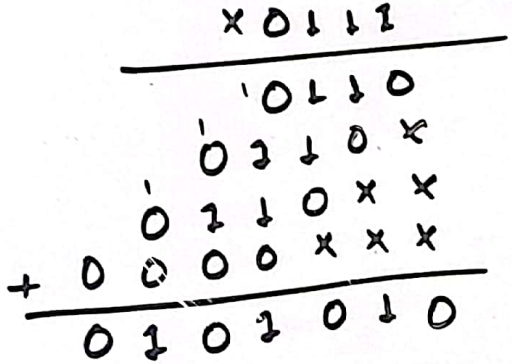
```
      X = 1101
    x Y = 1110
    ---------
        0000
       1101x
      1101xx
     1101xxx
+   1101xxx
-----------
1011010
```

A B which is true.

Soln:

M = Multiplicand = 0110 = X
C = 0 (carry)
A = 0000 (Accumulator)
B = Multiplier = 0111 = Y
n = no. of bits in B = 4

Flow chart:



There are 8 bits in result of AB, so we have to make this result of same length by adding 0 in MSB

So, we get. (A) 00101010 (B), which is true. #.

2016 pattern

Q16). Write steps for Booth's algorithm. Trace the RTL code for shift add multiplication algorithm for X = 0110 and Y = 0111.

	C	A	B	Count(n)	Remarks
					Initialization
i.	0	0000	0111	4	
		0110	0111	4	A ← A+M
ii.	0				shift Right CAB
	0	0011	0011	4	
	0	0011	0011	3	Count ← count - 1
		1001	0011	3	A ← A+M, C ← 0
iii.	0				shift Right CAB
	0	0100	1001	3	
	0	0100	1001	2	count ← count - 1
		1010	1001	2	A ← A+M, C ← 0
iv.	0				shift Right CAB
	0	0101	0100	2	
	0	0101	0100	1	Count ← Count - 1
					shift Right CAB
v.	0	0010	1010	1	
	0	0010	1010	0	Count ← Count - 1

Result in AB = 00101010

Sol^n Here,

M = Multiplicand = 4 = 0100

C = 0 (carry)

A = 0000 (Accumulator)

B = Multiplier = 0011

n = no. of bits in B = 4

2017 Fall

2(i) Trace the multiplication of (4) and (3) using RTL code of shift add multiplication algorithm.

	C	A	B	Count (n)	Remarc
i.	0	0000	0011	4	Initialization
ii.	0	0100	0011	4	A ← A+M, C ← 0
	0	0010	0001	4	shift Right CAB
	0	0010	0001	3	n ← n-1
iii.	0	0110	0001	3	A ← A+M, C ← 0
	0	0011	0000	3	shift Right CAB
	0	0011	0000	2	n ← n-1
iv.	0	0001	1000	2	shift Right CAB
	0	0001	1000	1	n ← n-1
v.	0	0000	1100	1	shift Right CAB
	0	0000	1100	0	n ← n-1

we get,

AB = 00001100

Result check:

```
  0100
x 0011
------
  0100
 0100x
0000xx
+0000xxx
--------
00001100
```

Milon


Believe in yourself and you can achieve anything.
GOOD LUCK WITH YOUR EXAMS